SmartFactory Predictive Quality Control: Comparing Classical Machine Learning and Neural Networks

Module: 6FTC2175 Advanced Machine Learning and Neural Networks — Coursework 1

Student: [Keith Phang] · [25034227]

Notebook and code: [https://github.com/PPingPong/AMLNN-CW1-SmartFactory]

Word count: approximately 1,520 words (excluding title block, headings, tables, figures, captions and references)

1. Introduction

SmartFactory wants to stop defective products before they reach customers. Part A of this report compares a Random Forest and an MLP on process-sensor data (vs a convolutional neural network (CNN) trained from scratch on product photographs in Part B). Part C recommends deployment.

The faulty case is the positive class everywhere. All model, hyper-parameter and threshold choices used training or validation data and each test set was used once. Models are judged on precision, recall, F1, PR-AUC and MCC (Matthews correlation coefficient) because accuracy deceives: always predicting "pass" scores 96.6% on the sensor data.

2. Part A — Sensor data: Random Forest vs MLP

2.1 Data and preprocessing

The AI4I 2020 dataset (Matzka, 2020) has 10,000 milling runs of which 339 (3.4%) end in machine failure. Identifiers and the five failure-mode flags were removed; target is "any flag = 1", so flags would leak answers. Power (torque × angular speed), temperature difference (process − air) and strain (tool wear × torque) encode documented failure mechanisms — each adds 0.06–0.08 cross-validated PR-AUC by itself, but together they push the Random Forest from 0.771 to 0.898. Data was split 70/15/15 with stratification and MLP's scaler fitted on training rows only.

2.2 Model selection

Six classical candidates from the module were compared by 5-fold cross validation on the training set (Figure 1). Naive Bayes scored worst (0.48) since independence assumption fails (torque and speed correlate at −0.88): RBF-SVM (0.63), single tree (0.64) also struggled; Random Forest (0.898 ± 0.045) and Gradient Boosting (0.904 ± 0.041) were statistically tied. Random Forest was chosen because it trains in parallel, is less sensitive to tuning than boosting and provides out-of-bag estimates and feature importances (Breiman, 2001). Out-of-bag scores flattened beyond 200 trees and a cross-validated grid search selected depth 8, minimum leaf size 8 and all features per split (CV PR-AUC 0.916 ± 0.023).

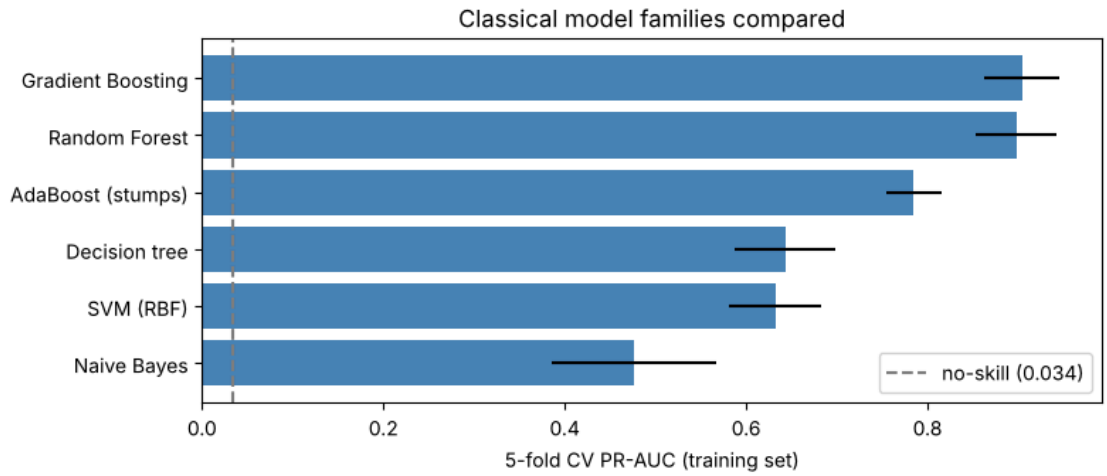


Figure 1. Five-fold cross-validated PR-AUC of six classical models (training set only).

The MLP has ReLU hidden layers and one sigmoid output. Binary cross entropy and Adam are used for training. A 4-unit network underfitted (validation PR-AUC 0.46). A 128–64 network overfitted: training PR-AUC rose to 0.94 but validation stalled at 0.75 (Figure 2). L2 (10^{-4}), dropout (0.3) and early stopping increased validation PR-AUC to 0.787 so this was selected model. Balanced class weights were worse (0.747). Both models' thresholds maximise F1 on validation.

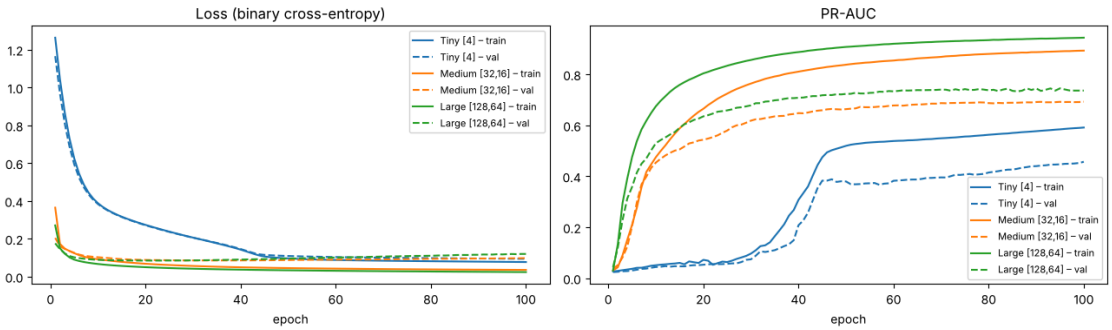


Figure 2. MLP capacity: the tiny network underfits; the large one overfits (train–validation gap).

2.3 Results and interpretation

Both models were scored on the same 1,500 held-out test rows (51 failures).

Model	Precision	Recall	F1	PR-AUC	MCC	Missed	False alarms	Train time
Random Forest	0.958	0.902	**0.929**	**0.937**	**0.927**	5	2	2.3 s
MLP	0.843	0.843	0.843	0.890	0.838	8	8	48 s

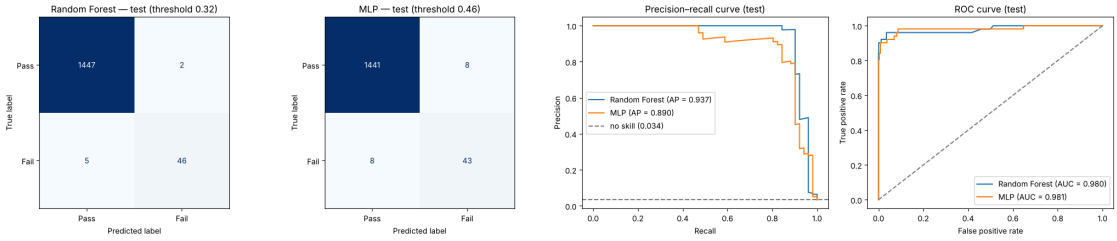


Figure 3. Test-set confusion matrices, precision–recall and ROC curves.

The Random Forest is better on every failure-class metric. In McNemar's paired test 10 rows were correct only for RF and 1 only for MLP (exact p = 0.012), so the difference in errors isn't random. Significance doesn't show operational importance; here the gap is 3 fewer missed failures and 6 fewer false alarms per 1500 runs. RF also won all five paired CV folds (PR-AUC 0.917 ± 0.040 vs 0.734 ± 0.114) and F1 didn't vary across seeds (MLP 0.830–0.851). With 51 test failures uncertainty remains: Wilson 95% intervals for recall overlap (0.79–0.96 vs 0.72–0.92).

Three reasons explain the result. First, the failure mechanisms are threshold rules (e.g. high wear with high torque) that axis-aligned splits capture directly; an MLP must approximate sharp corners with smooth ReLU combinations. Second, with only 237 training failures, bootstrap averaging controls variance better than regularising 9,857 weights, and the RF's lead grows when data is scarce (PR-AUC 0.62 vs 0.51 with 10% of the data). Third, permutation importance shows both models use the same physical drivers, so the MLP's deficit lies in fitting, not in what it looks at. It would benefit from far more data (Grinsztajn, Oyallon and Varoquaux, 2022).

3. Part B — Images: a CNN trained from scratch

3.1 Data audit and a leakage-free split

The casting dataset (Dabhi, 2020) has 7,348 grayscale impeller photographs in train and test folders. A second export of 1,300 images (512 × 512) was held back as an external test. Labels were consistent but the images had been augmented before splitting and have no casting IDs.

Each image was compared with every other under 48 rotations and mirrorings using a 256-bit perceptual hash. Opposite-label images (which must be different castings) were

rarely within 15 bits so a copy was ≤ 10 bits (Figure 4). **By this metric 47% of the author's test images had a copy in his training folder (64 of them byte-identical).** First model trained on that split scored about 99.9% (Partly through memory).

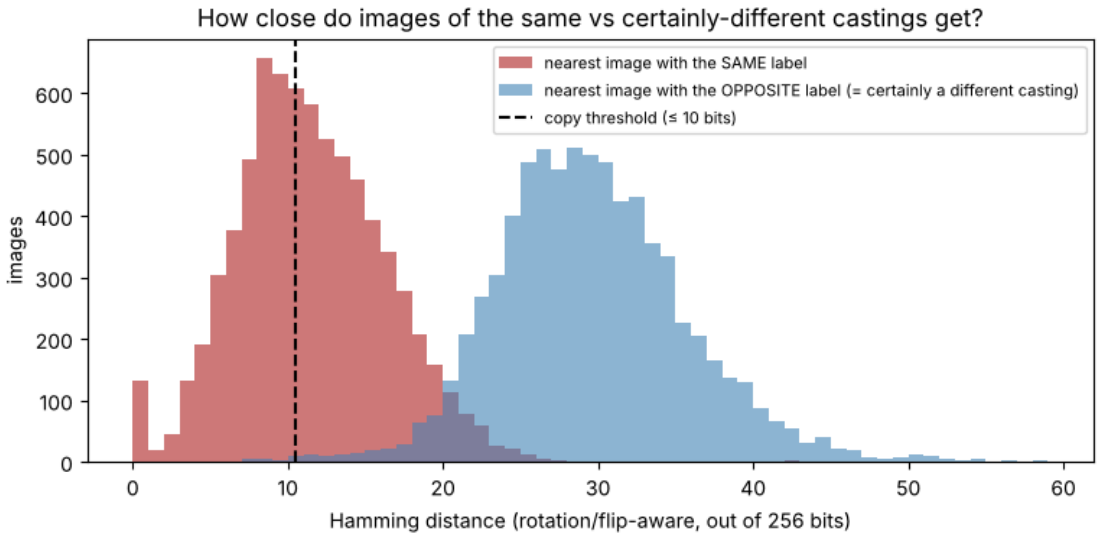


Figure 4. Calibrating the copy threshold: certainly-different castings (blue) are rarely within 10 bits; many same-label pairs (red) are.

All images were therefore pooled into 4,384 groups of copies and split 70/15/15 with StratifiedGroupKFold before training (5,038 / 1,290 / 1,020 images). No group spans two sets and the closest cross-set pair is 11 bits apart. Without real IDs leakage is reduced but not ruled out. Large single label groups also made validation less defect heavy (49%) than training (59%) or test (62%).

3.2 Architecture, loss and iteration

V1 is the module baseline: two 3 × 3 convolution and max-pooling blocks (32 and 64 filters), Flatten and one sigmoid unit (80,321 params; input 128×128). Convolution works for small local defects that can appear anywhere because shared filters detect them at any position. Loss is binary cross entropy negative log-likelihood of Bernoulli output so gradient wrt logit is just p − y not squared error on sigmoid which does saturate. Adam trained up to 20 epochs with early stopping and learning rate reduction on validation loss applied.

V2 adds random flips, ±18° rotations and dropout (0.3), with no extra parameters. Rotation is used because a casting can lie rotated on the conveyor. **V3** replaces Flatten with global average pooling and adds 128-filter blocks (100,993 parameters). This reduces dense head from 61,505 weights to 8,321 and widens the receptive field.

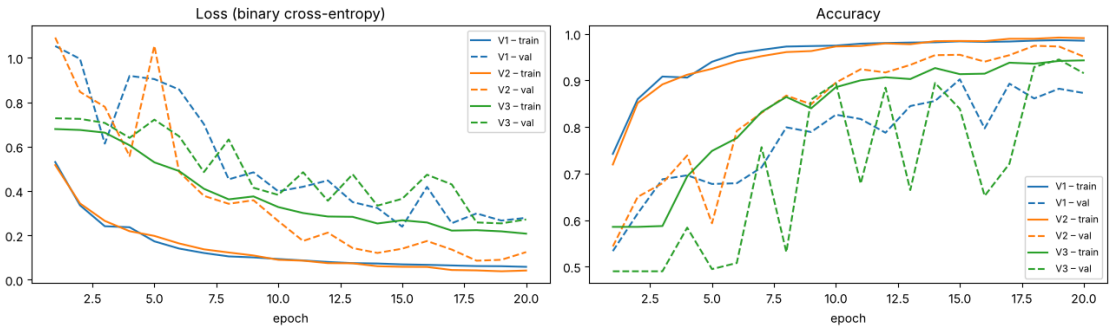


Figure 5. Learning curves on the grouped split (solid = training, dashed = validation).

The curves show the capacity story very well (Figure 5). **V1 overfits**: training loss is 0.07 but validation loss stalls near 0.24 because Flatten–Dense memorises castings. This gap was invisible on the leaky split. Augmentation in **V2 closes most of the gap** (validation loss 0.086). **V3 underfits**: training loss is still 0.22 after 20 epochs. V2 had the lowest validation loss and was selected before the test set was used.

3.3 Results, robustness and generalisation

Version	Grouped test F1	Grouped test MCC	Missed / false rejects (630 defective, 390 OK)	Second-export F1	Second-export MCC
V1 baseline	0.914	0.809	91 / 10	0.801	0.652
V2 augment + dropout (selected)	**0.968**	**0.922**	**35 / 4**	**0.847**	**0.679**
V3 GAP + extra block	0.932	0.825	48 / 37	0.831	0.515

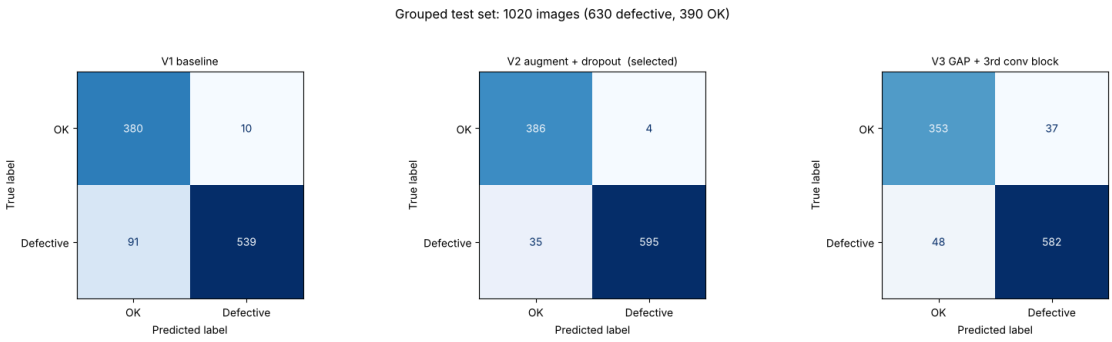


Figure 6. Grouped test set: almost all of V2's errors are missed defects.

On 1,020 unseen castings V2 got F1 0.968 (bootstrap 95% CI 0.958 -- 0.978) and recall 0.944 and test ranking matches validation. Three checks probe what it learned:

- **Shortcuts:** masking the background barely changes F1 (0.977 to 0.973 on validation). Grad-CAM (Selvaraju et al., 2017) places the evidence on the part (partly on rim defects) (Figure 7).
- **Imaging changes:** a 45° rotation has no effect. A 20% brighter image halves recall but darker or blurred images make the model flag almost every part.
- **Second export:** on 1279 images without copy in training pool F1 fell to 0.847 recall to 0.77 (Figure 8). Brightness, contrast and class balance match training images but backgrounds are brighter twice as variable and zoom varies.

The CNN does generalise well to unseen castings photographed the same way but not yet to a different imaging set-up.

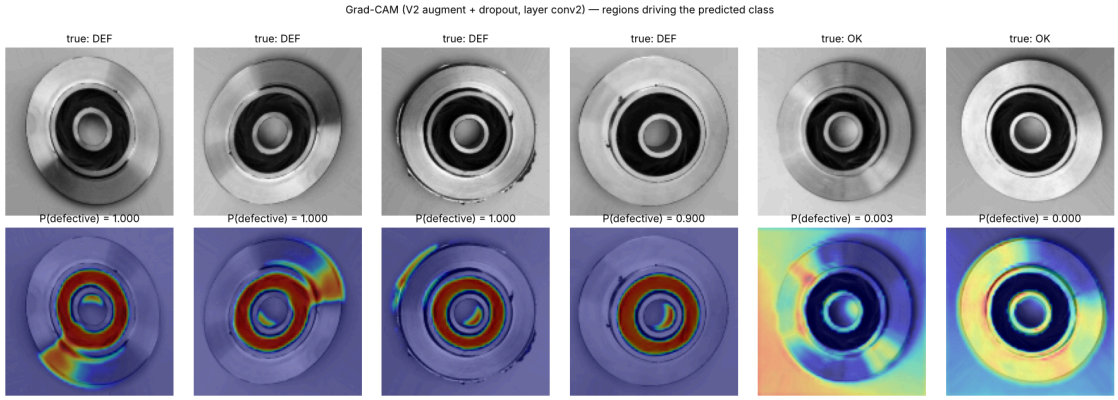


Figure 7. Grad-CAM for V2: evidence concentrates on the part, partly on rim defects.

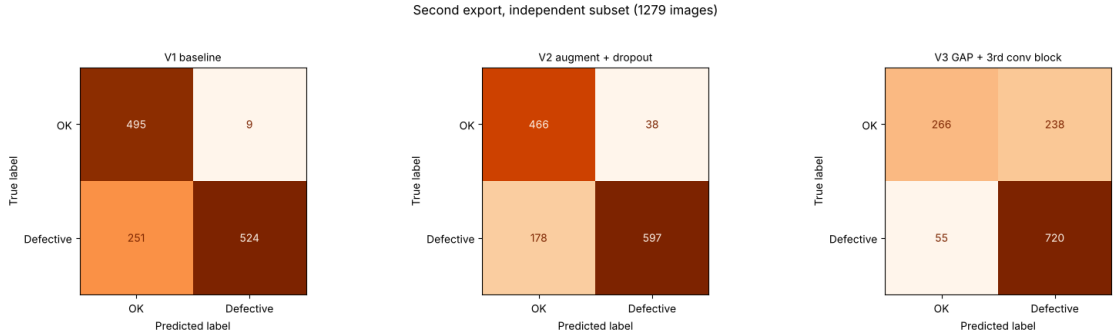


Figure 8. Second export (1,279 independent images): all versions lose recall.

4. Part C — Critical comparison and deployment

4.1 Comparing the three models

Dimension	Random Forest	MLP	CNN (V2)
Predictive performance	Best sensor model: F1 0.929, MCC 0.927; won all 5 CV folds	F1 0.843, MCC 0.838; varies by seed	F1 0.968 on unseen castings; 0.847 on a new export
Interpretability	Built-in importances and inspectable trees; drivers match failure physics	Black box; needs post-hoc permutation importance or SHAP	Black box; Grad-CAM and background masking give partial visual explanations
Data requirements	7,000 rows; degrades gracefully (PR-AUC 0.62 with 10% of data)	Needs more data (0.51 with 10%) and careful regularisation	Thousands of labelled images, ideally with casting IDs; near chance with 10–30% of the data in our runs
Training cost (2-core CPU)	2.3 s, no scaling	48 s, needs scaling	about 10 min per version; a GPU is practical
Inference and size	0.05 ms per run; 0.7 MB	0.04 ms; 0.04 MB	1.2 ms per image; 0.3 MB; needs a camera station
Main risk	Only 51 test failures; synthetic data	Instability across seeds	Silent errors when lighting or framing change

The models are trying to solve two different problems. The CNN sees defects in roughly balanced data while sensor models predict a 3.4% rare event. So CNN's higher F1 **doesn't mean it is a better model** and performance is read within each task.

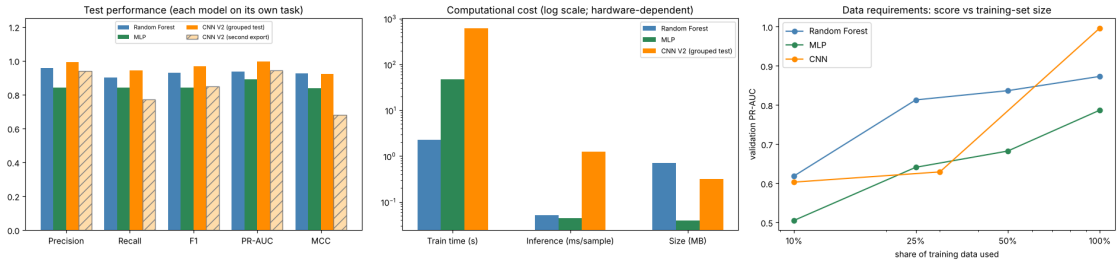


Figure 9. Left: test metrics, each on its own task (hatched = CNN on the second export). Centre: cost on a log scale. Right: validation PR-AUC against training-set size.

4.2 From metrics to manufacturing decisions

Thresholds should reflect what each error costs (a missed defect ships a faulty pump; false alarm costs an inspection). Without factory data the analysis assumes that a miss costs 20×

a false alarm for sensors and 50× for images. These ratios are only illustrative. Thresholds were chosen on validation data only.

- **CNN:** The cost-based threshold reduced missed defects on test sets from 35 to 8 for 9 extra false rejects. Sensitivity analysis (ratios 1–100) showed this choice holds for a ratio of 5 upward.
- **Random Forest:** The cost-based threshold was unstable (validation had only 51 failures) and multiplied false alarms. Instead, the model auto flags above F1 threshold and sends the next 5% most suspicious runs to human review (3 of 5 missed failures were recovered).
- **Images:** Reviewing 4.6% of castings found 31 of V2's 35 missed defects.

4.3 Recommendation

1. **Deploy the Random Forest** on the machine sensors as an early warning system (F1 threshold + 5% review band). It was the most accurate, stable, cheap explainable sensor model. Validate first on SmartFactory's own sensor history since AI4I is synthetic.
2. **Pilot the CNN (V2) in shadow mode** with inspectors next to it (recall threshold). It works great on castings photographed the same way but the second export and lighting results tell us it could silently miss a quarter of defects under a new setup. Fix the lighting and framing, retrain for brightness and scale augmentation, validate on new castings with part IDs before allowing automatic rejection.
3. **Do not deploy the MLP.** It was less accurate, less stable and less interpretable than the Random Forest here.

After deployment monitor image brightness, background and framing; sensors' input distributions; how often reviewers overturn the models. Re-validate whenever the machine, camera or product changes.

5. Conclusion

On sensor data a tuned Random Forest beat the regularised MLP on every check. On images a CNN trained from scratch with augmentation got F1 0.968 once a group-aware split removed leakage that had inflated scores to 99.9%. The lessons are methodological (check data before splitting, only select on validation data and judge models by cost of error). The open risk for the CNN is generalisation to new imaging conditions.

References

Abadi, M. et al. (2016) 'TensorFlow: a system for large-scale machine learning', 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 265–283.

Breiman, L. (2001) 'Random forests', Machine Learning 45(1), pp. 5–32.

Dabhi, R. (2020) Real-life industrial dataset of casting product. Kaggle. Available at: https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product.

Grinsztajn L., Oyallon E. and Varoquaux G. (2022) 'Why do tree-based models still outperform deep learning on typical tabular data?', Advances in Neural Information Processing Systems 35 (Datasets and Benchmarks Track).

Matzka, S. (2020) 'Explainable artificial intelligence for predictive maintenance applications', 2020 Third International Conference on Artificial Intelligence for Industries (AI4I), pp. 69–74. Dataset: UCI Machine Learning Repository, id 601.

McNemar, Q. (1947) 'Note on the sampling error of difference between correlated proportions or percentages' Psychometrika 12(2), pp. 153–157.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research 12(4), pp. 2825–2830.

Selvaraju, R.R. et al (2017) 'Grad-CAM: visual explanations from deep networks via gradient based localization', Proceedings of the IEEE International Conference on Computer Vision (ICCV), 618–626.

Wilson, E.B. (1927) 'Probable inference, law of succession and statistical inference', Journal of the American Statistical Association, 22(158), pp. 209–212.

AI tool declaration

AI tool used	Purpose of use
Claude (Anthropic)	Debugging the notebook code (error corrections). Compare it to the rubrics, to see missing criteria; which I reviewed, verified against my notebook outputs and edited. Without copying the entire output from Claude just making use of idea and suggestion